

Modelos Formais para Tradução

Gramática

39

- Consiste num conjunto de regras (*produções*) que definem *itens léxicos* e *combinações* desses para formar *sentenças válidas* em uma linguagem.
 - ▣ Uma gramática formal usa uma notação formal, isto é restrita e específica.
- Classes de gramáticas úteis para a teoria de compilação:
 - ▣ BNF (gramática livre de contexto),
 - ▣ Gramática regular.
- Uma linguagem é qualquer conjunto de strings (de tamanho finito) de caracteres escolhidos de um alfabeto fixo de símbolos finitos.

Modelos Formais para Tradução

Gramática Backus-Naur Form (BNF)

40

- Uma gramática BNF é composta de um conjunto *finito* de regras gramaticais BNF, descrevendo uma linguagem

- ▣ É uma *metalinguagem* ou seja uma linguagem usada para descrever outras linguagens.

- ▣ Gramática BNF é a definição formal da sintaxe de uma LP usando a notação BNF: *terminais*, ' $::=$ ', ' $<$ ', ' $>$ ' ou ' $|$ '.

$::=$

é definido como

$<ca>$

ca é uma categoria sintática, símbolo não terminal, definido em termos de outras entidades).

terminais são palavras primitivas (itens léxicos) da LP.

$|$

ou

Gramática BNF

Notações equivalentes

41

□ EBNF

[...] indica elementos opcionais.

[|] indica alternativas de escolha.

{...}* indica sequência arbitrária de instâncias (zero ou mais repetições) de elementos sintáticos.

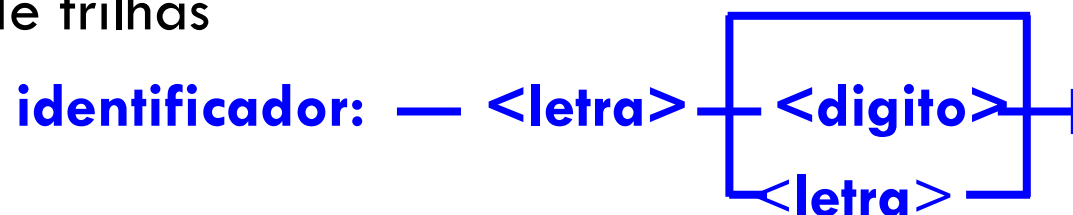
<identificador> ::= <letra>{<letra> | <dígito>}

<inteiro> ::= [+ | -]<dígito>{<dígito>}

Alguns extensões usam a notação {...}⁺ significando uma ou mais ocorrências do elemento sintático envolvido:

<inteiro> ::= [+ | -]{<dígito>}⁺

□ Diagrama de trilhas



Gramática BNF

Notações equivalentes

42

- Regra de produção

Obtida pela substituição de $::=$ pelo símbolo $\rightarrow$ e representação de não terminais como uma única letra maiúscula

Exemplo:

$$\langle X \rangle ::= \langle B \rangle \mid \langle C \rangle$$
$$X \rightarrow B \mid C$$

- Dada uma gramática, pode-se utilizar uma regra de substituição simples para gerar strings válidas na linguagem:

- ▣ substitua qualquer não terminal pela expressão no lado direito de uma regra de produção que o contenha no lado esquerdo.

$S \rightarrow SS \mid (S) \mid ()$ gera sequências de parênteses corretas.

P.ex.: $S \Rightarrow (S) \Rightarrow (SS) \Rightarrow (()S) \Rightarrow (()())$

Gramática BNF

Árvore de análise (parse)

43

- Uma string representa um programa sintaticamente válido em uma gramática BNF se passar, sem erro, por uma análise sintática, utilizando as regras da gramática. Essa análise gera uma árvore de parse.

Exemplo:

$x=y+5*(z+x);$ e as regras

$\langle \text{atribuição} \rangle ::= \langle \text{variável} \rangle = \langle \text{ea} \rangle$

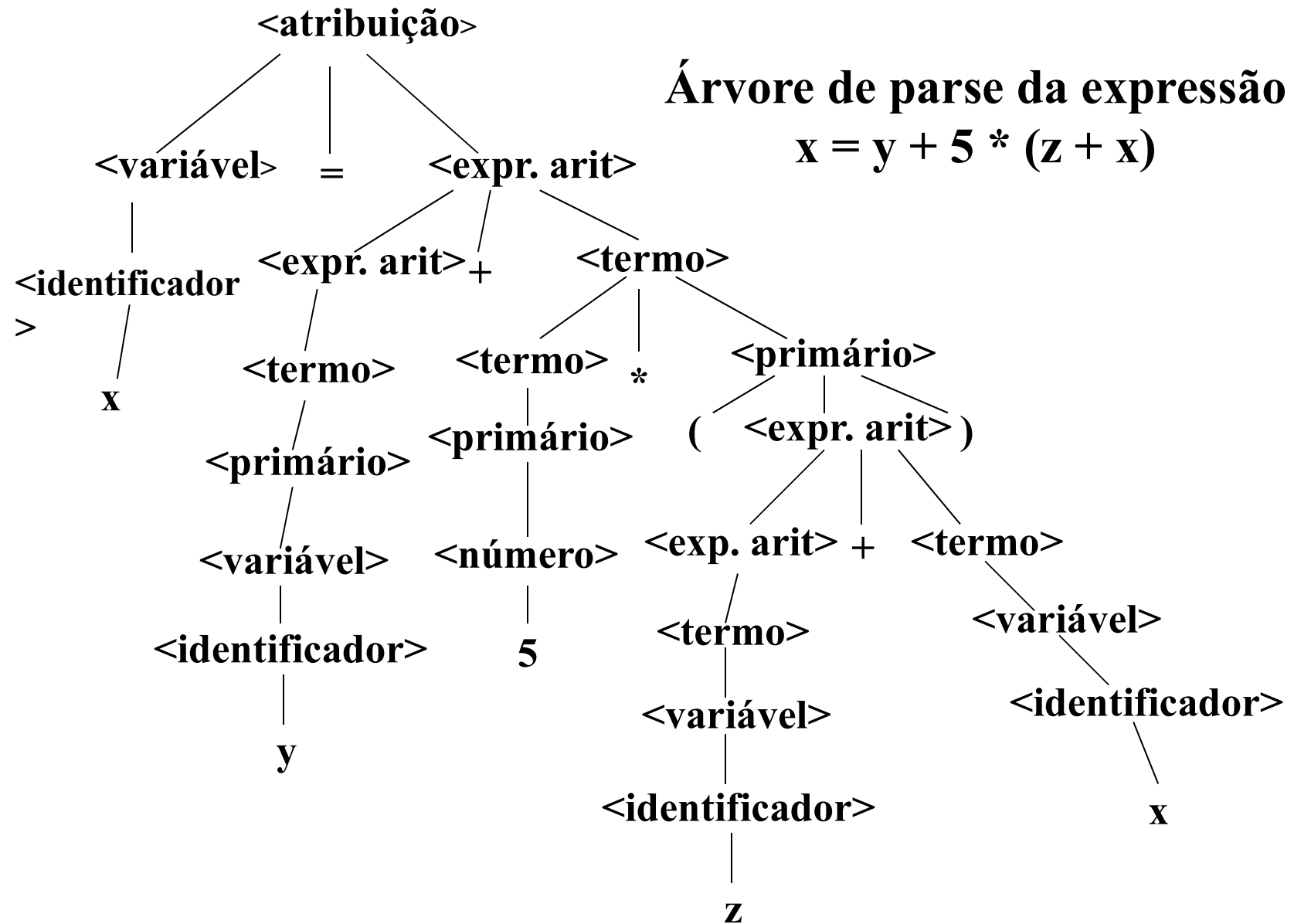
$\langle \text{ea} \rangle ::= \langle \text{term} \rangle \mid \langle \text{ea} \rangle + \langle \text{term} \rangle \mid \langle \text{ea} \rangle - \langle \text{term} \rangle$

$\langle \text{term} \rangle ::= \langle \text{primário} \rangle \mid \langle \text{term} \rangle * \langle \text{primário} \rangle \mid$
 $\langle \text{term} \rangle / \langle \text{primário} \rangle$

$\langle \text{primário} \rangle ::= \langle \text{variável} \rangle \mid \langle \text{número} \rangle \mid (\langle \text{ea} \rangle)$

$\langle \text{variável} \rangle ::= \langle \text{identificador} \rangle [(\langle \text{lista de subscritos} \rangle)]$

$\langle \text{lista de subscritos} \rangle ::= \langle \text{ea} \rangle \mid \langle \text{lista de subscritos} \rangle, \langle \text{ea} \rangle$



Gramática BNF

Limitações sintáticas

45

- A estrutura da BNF é simples e muito poderosa mas não consegue expressar regras sintáticas com *dependência contextual* do tipo:
 - ▣ o mesmo identificador não pode ser declarado mais de uma vez no mesmo bloco,
 - ▣ todo identificador deve ser declarado em um bloco envolvendo o ponto onde é usado,
 - ▣ um array deve ser referenciado com o mesmo número de subscritos com o qual é definido.
- BNF é uma gramática livre de contexto, isto é o lado esquerdo de uma regra só pode conter um símbolo.

Modelos Formais para Tradução

Autômato de estados finitos

46

- Na fase de análise léxica o programa fonte é transformado numa sequência de tokens (itens léxicos).
- Tokens podem ser reconhecidos por um modelo de máquina de estados também chamada autômato de estados finitos.
- Formalmente um FSA = (E, e_i, E_f, A_e, A_o) , onde:
 - E = conjunto finito de estados (nós no grafo),
 - e_i = estado inicial (um nó no grafo),
 - E_f = conjunto de estados finais, $E_f \subset E$,
 - A_e = alfabeto de entrada (rótulos para os arcos),
 - A_o = conjunto de arcos orientados ligando elementos de E .

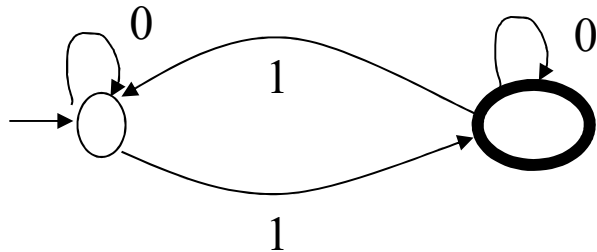
Cada nó pode ter zero ou mais arcos de saída, incluindo múltiplos arcos com o mesmo rótulo. Cada nó pode ter zero ou mais arcos de chegada.

Autômato de Estados Finitos

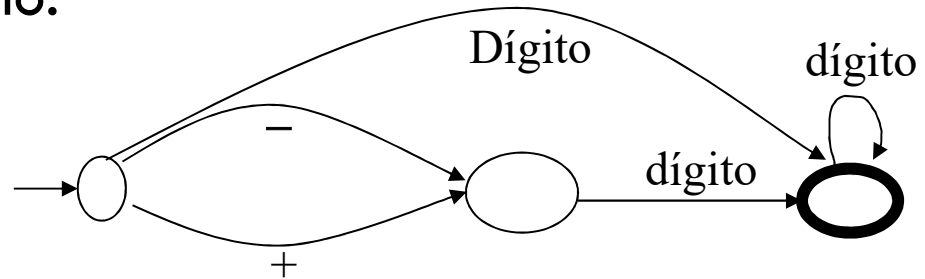
Determinístico e não determinístico

47

- Determinístico: FSA *sem* repetição de arcos de *saídas* com o *mesmo rótulo* em um mesmo nó.

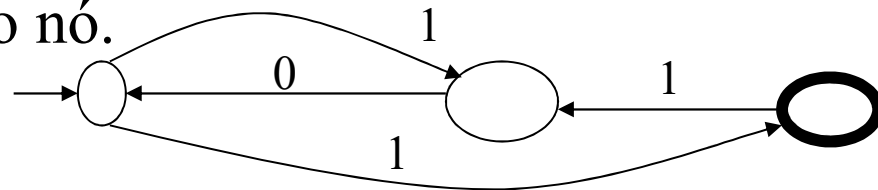


Reconhecedor de número com número ímpar de 1s.



Reconhecedor de: $[+|-] \langle \text{dígito} \rangle \{ \langle \text{dígito} \rangle \}^*$

- Não determinístico: FSA *com* repetição de arcos de *saída* com o *mesmo rótulo* em um mesmo nó.



Uma string é aceita pelo FSA não determinístico se existe **algum** caminho do nó inicial para o nó final, usando a string como entrada.

10101

Autômato de Estados Finitos

Gramáticas regulares

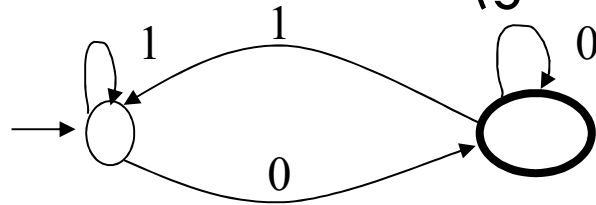
48

- Possuem regras da forma:
 $\langle \text{não-terminal} \rangle ::= \text{terminal} \langle \text{não-terminal} \rangle \mid \text{terminal}$
- São casos especiais de gramática BNF.
- O conjunto de linguagens aceitas por FSA é equivalente às linguagens geradas por gramáticas regulares.

Exemplo: $\langle B \rangle ::= 0\langle B \rangle \mid 1\langle B \rangle \mid 0$

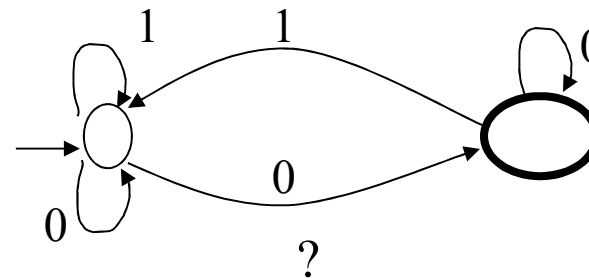
a) a regra de produção $B \rightarrow 0B \mid 1B \mid 0$ gera binários pares

b) FSA determinístico (gera/reconhece pares)



10101

Determinístico



Autômato de Estados Finitos

Expressões regulares

49

- Terceira forma de definição de linguagens (equivalente a FSA e a gramática regular).
- $\langle \text{er} \rangle ::= \text{terminal} \mid \langle a \rangle \vee \langle b \rangle \mid \langle a \rangle \langle b \rangle \mid (\langle a \rangle) \mid \{ \langle a \rangle \}^*$

onde:

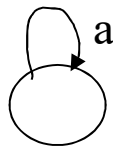
- um símbolo terminal é uma expressão regular,
- $a \vee b$ é a alternância das expressões regulares a ou b ,
- ab é a concatenação das expressões regulares a e b ,
- a^* , fecho de Kleene, representa zero ou mais repetições de a (i.e. $\varepsilon, a, aa, aaa, \dots$).

Autômato de Estados Finitos

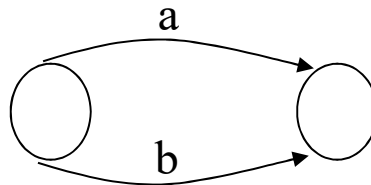
Expressões regulares

50

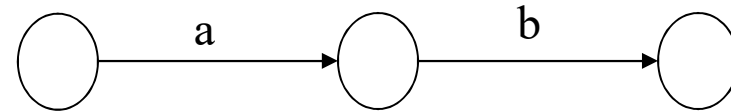
- Converter um FSA para uma expressão regular nem sempre é óbvio.
- Converter expressões regulares em um FSA é direto, com a aplicação das estruturas:



Fecho de Kleene: a^*



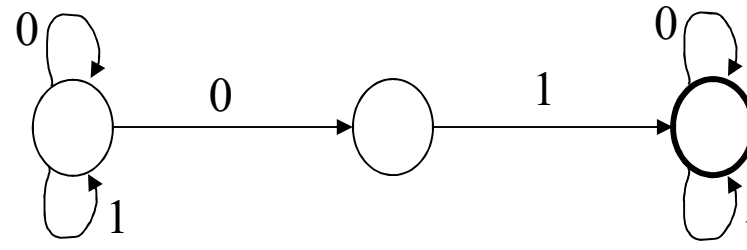
Alternação: $a \vee b$



Concatenação: $a b$

Exemplo:

$(0 \vee 1)^* 0 1 (0 \vee 1)^*$



Modelos Formais para Tradução

Autômatos pushdown (PDA)

51

- Gerar strings em uma linguagem $\Rightarrow$ *gramáticas BNF*.
- Reconhecer strings na linguagem $\Rightarrow$ *autômato pushdown*
- PDA é um FSA com uma pilha associada.
- Os movimentos do PDA são os seguintes:
 1. um símbolo de entrada é lido e o topo do stack também é lido;
 2. os dois símbolos são comparados. O PDA entra em um novo estado e escreve zero ou mais símbolos na pilha;
 3. a aceitação de uma string ocorre se a pilha ficar vazia (ou de forma equivalente, se o PDA atingir um estado final).
- Para reconhecer $a^n b^n$, empilhe a . Para cada entrada b , desempilhe um a . A string é aceita se o término dos b ocorrer com pilha vazia.

Autômatos Pushdown

PDA não determinístico

52

- PDA não determinístico é um autômato que possui mais de um estado com a mesma entrada.

Uma string é aceita se existe uma sequência possível de movimentos que aceita a string.

- PDA determinístico e não determinístico são DIFERENTES

Palindromes $S \rightarrow 0S0 \mid 1S1 \mid 2$ são reconhecidas pelo PDA determinístico, com os seguintes movimentos:

1. empilhe todos os 1 e 0 lidos. 101020101
2. mude de estado com 2.
3. desempilhe se cada entrada for igual ao topo da pilha.
4. a string é aceita se encerrar a entrada com stack vazia.

Autômatos Pushdown

PDA não determinístico: exemplo

53

$S \rightarrow OS0 \mid 1S1 \mid 0 \mid 1$ gera **0110**10110 ?

$S \Rightarrow OS0 \Rightarrow 0 \ 1S1 \ 0 \Rightarrow 01 \ 1S1 \ 10 \Rightarrow 011 \ OS0 \ 110 \Rightarrow 0110 \ 1 \ 0110$

PDA não determinístico reconhece essa string? Somente se apostar (chutar) onde está o meio!

stack	meio?	compara stack com
ε	0	11010110
0	1	1010110
01	1	010110
011	0	10110
0110	1	0110 correto!

- PDA não determinístico reconhece qualquer gramática livre de contexto.

Modelos Formais para Tradução

Algoritmos eficientes de parsing

54

- Uma gramática descreve a estrutura do programa de um modo **top-down** especificando todo o programa, depois os subprogramas, comandos, declarações, etc.
- O parser atua no sentido **bottom-up** para gerar uma árvore de parse a partir de uma sequência de tokens, representando o fonte:
 - a estrutura é construída analisando a entrada (tokens) da esquerda para a direita (LR), à medida que os tokens são lidos.**
 - a sequência de tokens pode não constituir um programa válido!**
- Cada tipo de gramática formal está relacionado com um tipo de autômato.

Algoritmos Eficientes de Parsing

Estratégia Geral

55

- Um *autômato* é uma máquina abstrata capaz de ler uma fita com uma sequência de caracteres e produzir uma fita de saída com outra sequência de caracteres.
- Se a BNF for ambígua o *autômato* é *não determinístico*:
existem opções de movimento e o autômato deverá apostar qual é o mais apropriado em um certo instante.
- Tradução de linguagem requer autômato determinístico.
- Gramática regular: sempre existe um autômato determinístico equivalente.
- Gramática BNF não ambígua: foi desenvolvido o *parsing recursivo descendente* para reconhecer sentenças válidas.

Algoritmos Eficientes de Parsing

Parsing recursivo descendente

56

- **Gramática BNF não ambígua**, reconhecida por PDA determinístico, **pode ser descrita por gramática LR** (left to right), desenvolvida pelo Knuth.
SLR (simple LR) e LALR (look ahead LR): subclasses de gramáticas LR com algoritmos de parsing eficientes.
- LP atuais usam gramáticas SLR, LR ou LL para que os parsers possam ser obtidos automaticamente via YACC.
- LR(k): gramáticas que olham k símbolos a frente, da esquerda para a direita, para tomar decisão de parsing.
LR(1): gramáticas que precisam olhar apenas um símbolo a frente para decidir qual o elemento sintático encontrado.

Parsing recursivo descendente

57

- **Parsing** é o processo de construir uma árvore de análise para um dado string de entrada:
 - em geral não analisa tokens.
 - um **parser recursivo descendente** constrói uma árvore de parse usando uma abordagem top-down.
 - **cada não terminal na gramática** tem um subprograma associado a ele que analisa as formas sentenciais que o não terminal pode gerar.
 - os subprogramas de parsing recursivo descendente não podem ser construídos a partir de gramáticas recursivas à esquerda.

Parsing recursivo descendente

Exemplo: expressão aritmética

58

$\langle \text{ea} \rangle ::= \langle \text{term} \rangle \mid \langle \text{ea} \rangle + \langle \text{term} \rangle \mid \langle \text{ea} \rangle - \langle \text{term} \rangle$

- **representação equivalente mais adequada para parsing.**

$\langle \text{ea} \rangle ::= \langle \text{term} \rangle \{ [+ \mid -] \langle \text{term} \rangle \}^*$

$\langle \text{term} \rangle ::= \langle \text{primary} \rangle \{ [* \mid /] \langle \text{primary} \rangle \}^*$

- Se reconhece um termo, se seguido de + ou -, reconhece um novo termo e assim sucessivamente.

- Convenções

nextchar: 1º caracter do não terminal.

getchar: ler um caracter da entrada.

Identifier: scanner p/ ler identificador.

Number: scanner p/ ler número.

procedure AssingStmnt

begin

Variable;

if nextchar $\neq$ '=' then **error**

else begin nextchar := getchar;

Expression **end**

end;

procedure Expression

begin

Term;

while ((nextchar='+') or (nextchar='-'))

do begin nextchar:=getchar;

Term **end**

end;

Parsing recursivo descendente

Exemplo: expressão aritmética

59

procedure Term;

begin

Primary;

**while (nextchar = '*') or (nextchar = '/')
do begin nextchar := getchar; Primary
end**

end;

procedure Variable;

begin

Identifier;

if nextchar = '(' then begin

nextchar := getchar; SubList;

if nextchar = ')' then nextchar :=

getchar

else Error /* falta ')' */

end

end;

procedure Primary;

begin

if nextchar = letter then Variable

else if nextchar = digit then Number

else if nextchar = '(' then

begin

nextchar := getchar;

Expression;

**if nextchar = ')' then nextchar
:= getchar**

else Error /* falta ')' */

end

else Error /* falta '(' */

end;